

Lab Activity #6 - Several types of migration

In this assignment we will apply several types of migration to the simple.su data. These types of migration represent many of those that are commonly used in industry.

1.1 Different types of “velocity”

All seismic migrations require a background wave speed (velocity) profile. However, we must be very careful when addressing the term “velocity.” The actual wave speeds in the subsurface are called interval velocities, and most likely are going to be a function of position.

However, the types of velocities that we often encounter, such as those obtained from velocity analysis are stacking velocity, also known as the NMO velocity, which is approximately the RMS (root mean squared) velocity. Such velocities are often expressed as *velocity as a function of time*.

Having a velocity that is a function of time may seem strange, at first, but imagine that you have a seismic section, with several strong seismic horizons. These strong arrivals must represent relatively large impedance contrasts. If we had a set of well logs to go with the seismic data, then we could identify those rock units with the strong impedance contrasts. We could build a collection of velocities chosen from the well log, and assign the times from the arrival time on the seismic section and thus we would define a $v(t)$. If we have $v(t)$ for specific locations in the rock volume we are investigating we could assemble a $v(t, x)$ or a $v(t, x, y)$ through some sort of interpolation.

1.1.1 Velocity conversion $v_{rms}(t)$ to $v_{int}(t)$

The conversion between these two is given by the Dix equation

$$v_{int} = \frac{(t_2 v_{rms2}^2 - t_1 v_{rms1}^2)^{1/2}}{(t_2 - t_1)^{1/2}}$$

where V_{int} is the interval velocity, t_1 is the travelttime to the first reflector, t_2 is the travelttime to the second reflector, V_{rms1} is the root-mean-squared velocity of the first reflector, and V_{rms2} is the root-mean-squared velocity of the second reflector. The RMS (root mean squared) part comes from the fact that we are effectively averaging and taking the square root of squares going from RMS to interval velocity.

Often the output of a migration is horizontal position versus “migrated time.” This implies that the input velocity is expressed as $v(t)$, rather than $v(X, Z)$. We may have interval or rms velocities as a function of time, e.g. $v_{int}(t)$ or $v_{rms}(t)$. This may seem strange at first, but if we have $v(t)$ profile, then $v(Z)$ is obtained by stretching time to depth.

In SU a program called `velconv` performs many simple conversions between interval and rms velocities. Type `velconv` with no options to see the selfdoc of the program

```
$ velconv
```

VELCONV - VELOCITY CONVERSION

`velconv <infile >outfile intype= outtype= [optional parameters]`

Required Parameters:

<code>intype=</code>	input data type (see valid types below)
<code>outtype=</code>	output data type (see valid types below)

Valid types for input and output data are:

<code>vintt</code>	interval velocity as a function of time
<code>vrms</code>	RMS velocity as a function of time
<code>vintz</code>	velocity as a function of depth
<code>zt</code>	depth as a function of time
<code>tz</code>	time as a function of depth

Optional Parameters:

<code>nt=all</code>	number of time samples
<code>dt=1.0</code>	time sampling interval
<code>ft=0.0</code>	first time
<code>nz=all</code>	number of depth samples
<code>dz=1.0</code>	depth sampling interval
<code>fz=0.0</code>	first depth
<code>nx=all</code>	number of traces

Example: "`intype=vintz outtype=vrms`" converts an interval velocity function of depth to an RMS velocity function of time.

Notes: `nt`, `dt`, and `ft` are used only for input and output functions

of time; you need specify these only for vintt, vrmst, orzt. Likewise, nz, dz, and fz are used only for input and output functions of depth.

The input and output data formats are C-style binary floats.

1.2 Stolt or (f, k) -migration

To migrate the simple data in the computer we first begin with Stolt migration. Stolt's method, published in 1978, is a migration by Fourier transform and is often called (f, k) migration. If we consider migration to be a "shifting of data," which is expressed as a signal processing technique, then we may consider that shifting to be done as a filtering process. The data are shifted both temporally and spatially, suggesting that the filter doing the shifting must be a filter that operates in both the frequency and wavenumber domain. If the velocity function is variable (with time) the Stolt method accounts for this by applying a stretch to the data prior to the filtering operation.

To get the shifting correct, Robert Stolt based his "shifting filter" on an integral equation representation of the wave equation, and made use of the *fast Fourier transform* algorithm for speed. To handle variable wavespeed Stolt introduced a "stretch," much as we scaled the time section to appear interchangeable with depth.

1.2.1 Stolt migration of the Simple model data

Here we apply Stolt migration to the simple.su data. Copy the dataset into your local working directory via:

```
$ cd ESC
$ mkdir Temp2
$ pwd (you should be in ESC)
$ cp /data/Data2/simple.su Temp2
```

changing your working directory to Temp2 via:

```
$ cd Temp2
```

You may view the data via:

```
$ suxwigg < simple.su xcur=3 title="Simple"
```

The Stolt migration is performed via the program sustolt. To see the self documentation for sustolt type:

```
$ sustolt
```

SUSTOLT - Stolt migration for stacked data or common-offset gathers

sustolt <stdin >stdout cdpmin= cdpmax= dxcdp= noffmix= [...]

Required Parameters:

cdpmin	minimum cdp (integer number) for which to apply DMO
cdpmax	maximum cdp (integer number) for which to apply DMO
dxcdp	distance between adjacent cdp bins (m)

Optional Parameters:

noffmix=1	number of offsets to mix (for unstacked data only)
tmig=0.0	times corresponding to rms velocities in vmig (s)
vmig=1500.0	rms velocities corresponding to times in tmig (m/s)
...	

Among the many parameters, those that are required are cdpmin, cdpmax, dxcdp, vmig and tmig. Note: the type of velocities, vmig, that this program requires are RMS (or stacking) velocities as a function of time! To see the range of the cdps in the data, type

```
$ surange < simple.su
80 traces:
trac 1 80 (1 - 80)
cdp 1 80 (1 - 80)
trid 1
ns 501
dt 4000
```

Thus, cdpmin=1 and cdpmax=80. The data were made with a 40m spacing, and with a velocity in the first medium of 2000m/s. Applying sustolt

```
$ sustolt < simple.su cdpmin=1 cdpmax=80
dxcdp=40 vmig=2000 tmig=0.0 > stolt.simple.su
```

We may view the output of Stolt migration via

```
$ suxwigb < stolt.simple.su xcur=3 title="Stolt migration of simple data" &
```

The data have been chosen to be too sparsely sampled spatially to be considered “good.” This choice was made on purpose to accentuate the artifacts on the data. These artifacts are known by the terms diffraction smiles, migration impulse responses, or endpoint contributions. The fact that the data are sparse makes the migration operator see the individual arrivals as single spikes, resulting in impulse responses on the output. There is insufficient coverage.

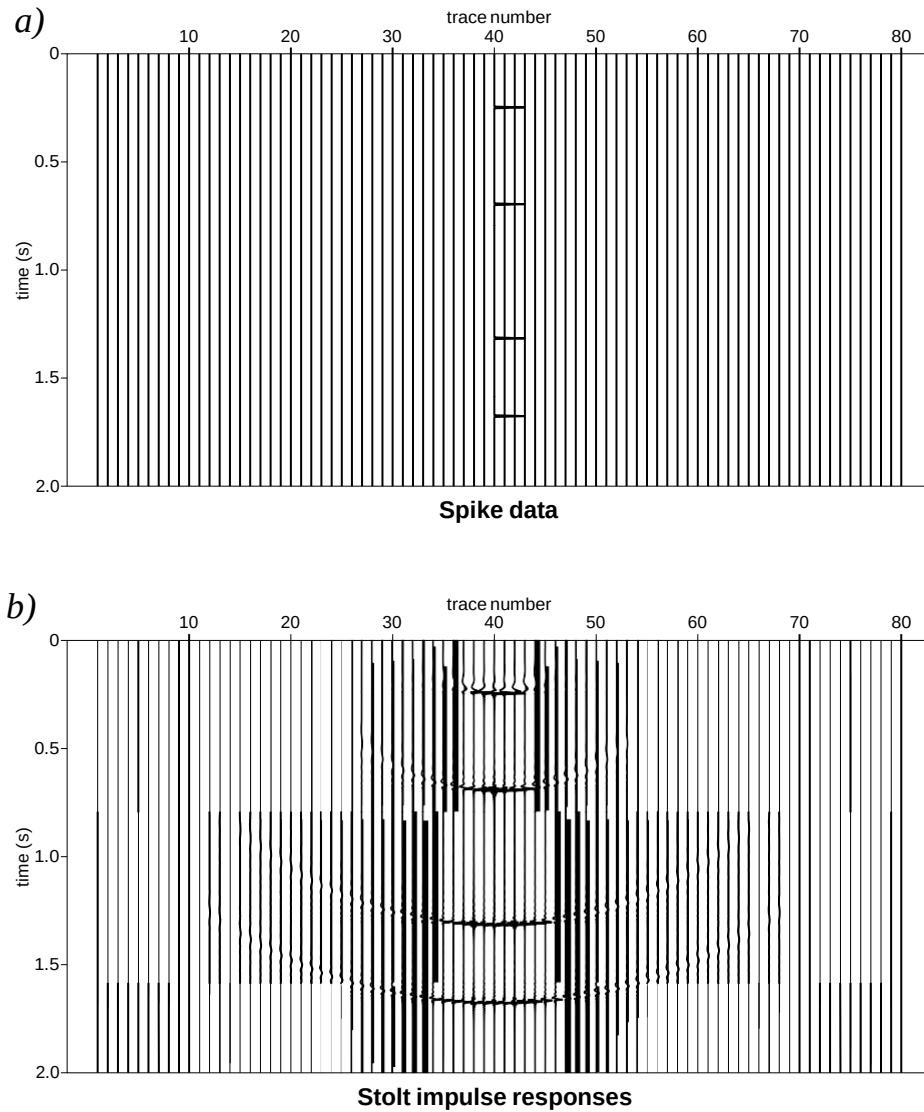


Figure 6.1: a) Spike data, b) the Stolt migration of these spikes. The curves in b) are *impulse responses* of the migration operator, which is what the Hagadoorn method were approximating. Not only do the curves represent every point in the medium where the impulses could have come from, the amplitudes represent the strength of the signal from that respective location.

Diffraction smiles as impulse responses

An effective way of thinking about many noise artifacts on migrated data is to note that migration operators expect smooth input. If the input is not smooth, such as if there is a noise spike or an abrupt termination of the data, this is similar to putting an impulse into the algorithm. The result is an *impulse response*.

By applying `suspike` we may make data consisting only of 4 nonzero data values (spikes). The data are otherwise the same number of samples (501) and the same number of traces (80) as `simple.su`

```
#!/bin/sh
```

```
$ suspike nspk=4 nt=501 ntr=80 dt=.004 ix1=40 it1=63 ix2=40 \
          it2=175 ix3=40 it3=40 it3=330 ix4=40 it4=420 \
          | sushw key=cdp a=1 b=1 > spike_simple.su
```

and we apply `sustolt` with the same parameters as we used on the `simple.su` data

```
$ sustolt < spike_simple.su vmig=2000 tmig=0.0
          dxcdp=40 cdpmin=1 cdpmax=80 > stolt.spike.su
```

shown in Figure 6.1

1.3 Gazdag or Phase-shift migration

In 1978 Jeno Gazdag published a type of migration that also makes use of the shift theorem. In this class of method, the model is discretized in such a way that the vertical direction z is considered to be preferred. The data are Fourier transformed, and the migration is applied as a phaseshift, to each wavenumber. The data are then inverse Fourier transformed.

The program `sugazmig`

```
$ sugazmig
```

SUGAZMIG - SU version of Jeno GAZDAG's phase-shift migration
for zero-offset data, with attenuation Q .

```
sugazmig <infile >outfile vfile= [optional parameters]
```

Optional Parameters:

dt=from header(dt) or .004	time sampling interval
dx=from header(d2) or 1.0	midpoint sampling interval

ft=0.0 first time sample

ntau=nt(from data) number of migrated time samples

$\text{dtau}=\text{dt}(\text{from header})$ migrated time sampling interval
 $\text{ftau}=\text{ft}$ first migrated time sample
 $\text{tmig}=0.0$ times corresponding to interval velocities in vmig
 $\text{vmig}=1500.0$ interval velocities corresponding to times in tmig
 ...

Try the Gazdag migration of the `simple.su` data via:

```
$ sugazmig < simple.su dx=40 vmig=2000 tmig=0.0 > gaz.simple.su
```

Note: the velocities, **`vmig`, here are interval velocities as a function of time.**

You will notice that you could do several Stolt migrations in the time that it takes to do a single Gazdag.

By the application of a slightly different formulation of phase-shift migration, the performance, as well as the accuracy at steeper dips of Gazdag migration can be improved. The result is Dave Hale's `sumigps`.

```
$ sumigps
```

SUMIGPS - MIGration by Phase Shift with turning rays

```
sumigps <stdin >stdout [optional parms]
```

Required Parameters:

None

Optional Parameters:

$\text{dt}=\text{from header}(\text{dt})$ or `.004` time sampling interval
 $\text{dx}=\text{from header}(\text{d2})$ or `1.0` distance between successive cdp
 $\text{ffil}=0,0,0.5/\text{dt},0.5/\text{dt}$ trapezoidal window of frequencies to migrate
 $\text{tmig}=0.0$ times corresponding to interval velocities in vmig
 $\text{vmig}=1500.0$ interval velocities corresponding to times in tmig
 $\text{vfile}=\text{binary}$ (non-ascii) file containing velocities $v(t)$
 $\text{nypad}=0$ number of cdp to pad with zeros before FFT
 $\text{ltaper}=0$ length of linear taper for left and right edges
 $\text{verbose}=0$ `=1` for diagnostic print

...

Try an improved phase-shift migration:

```
$ sumigps < simple.su dx=40 vmig=2000 tmig=0.0 > ps.simple.su
```

1.4 Claerbout finite-difference migration

Another “backpropagation” approach was taken by Jon Claerbout in 1970 via finite-difference solution of a one-way approximate wave equation. This is a reverse-time finite difference migration, but it is not the exact wave equation. The “15 degree” part refers to the angle of a cone about vertical, within which the travelttime behavior of the migration is sufficiently similar to that of the standard wave equation as to yield an acceptable result. Other approximations for increasingly larger angles have been created. An implementation of a version of this finite-difference migration that gives the user a choice of different “angles of accuracy” is sumigfd

SUMIGFD - 45-90 degree Finite difference migration for zero-offset data.

```
sumigfd <infile >outfile vfile= [optional parameters]
```

Required Parameters:

nz= number of depth samples
dz= depth sampling interval
vfile= name of file containing velocities
(see Notes below concerning format of this file)

Optional Parameters:

dt=from header(dt) or .004 time sampling interval
dx=from header(dx) or 1.0 midpoint sampling interval
dip=45,65,79,80,87,89,90 Maximum angle of dip reflector
...

Try:

```
$ cp /data/Data2/vel.fdmig.simple ESC/Temp2  
$ cd ESC/Temp2  
$ sumigfd < simple.su dx=40 dz=12 nz=150 vfile=vel.fdmig.simple > fd.simple.su
```

Note: the velocity file (vfile) expects interval velocities as a function of (x, z), where x is taken as the fast dimension in this file.

1.5 Ristow and Ruhl's Fourier finite-difference migration

A hybridization between phase-shift migration and finite-difference migration known as “Fourier finite difference” was published in 1994 by D. Ristow and T. Ruhl. This type of migration is implemented in sumigffd

SUMIGFFD - Fourier finite difference migration for zero-offset data. This method is a hybrid migration which combines the advantages of phase shift and finite difference migrations.

sumigffd <infile >outfile vfile= [optional parameters]

Required Parameters:

nz= number of depth samples
dz= depth sampling interval
vfile= name of file containing velocities

Optional Parameters:

dt=from header(dt) or .004 time sampling interval
dx=from header(dx) or 1.0 midpoint sampling interval
ft=0.0 first time sample
fz=0.0 first depth sample

...

Try:

```
$ cp /data/Data2/vel.fdmig.simple ESC/Temp2
$ cd ESC/Temp2
$ sumigffd < simple.su dx=40 dz=12 nz=150
                                vfile=vel.fdmig.simple > ffd.simple.su
```

Stoffa's split-step migration

Another algorithm, known as the “split-step” algorithm, developed by P. Stoffa, et al in 1990 is an extension of this idea with sumigsplit

SUMIGSPLIT - Split-step depth migration for zero-offset data.

sumigsplit <infile >outfile vfile= [optional parameters]

Required Parameters:

nz= number of depth samples
dz= depth sampling interval
vfile= name of file containing velocities

Optional Parameters:

dt=from header(dt) or .004 time sampling interval

dx=from header(d2) or 1.0	midpoint sampling interval
ft=0.0	first time sample
fz=	first depth sample
...	

Try:

```
$ cp /data /Data2/vel.fdmig.simple ESC/Temp2
$ cd ESC/Temp2
$ sumigsplit < simple.su dx=40 dz=12 nz=150
                                vfile=vel.fdmig.simple > split.simple.su
```

1.6 Gazdag Phase-shift Plus Interpolation migration

A problem with the original Gazdag phaseshift migration is that it did not handle lateral velocity variation well. An approach called “phase shift plus interpolation” (PSPI) was developed by Jenő Gazdag in 1984 that partially alleviates this problem. In SU this is implemented as `sumigpspi`

SUMIGPSPI - Gazdag’s phase-shift plus interpolation migration
for zero-offset data, which can handle the lateral
velocity variation.

`sumigpspi <infile >outfile vfile= [optional parameters]`

Required Parameters:

nz=	number of depth samples
dz=	depth sampling interval
vfile=	name of file containing velocities

(Please see Notes below concerning the format of vfile)

Optional Parameters:

dt=from header(dt) or .004	time sampling interval
dx=from header(d2) or 1.0	midpoint sampling interval
...	

Try:

```
$ sumigpspi < simple.su dx=40 dz=12 nz=150
                                vfile=vel.fdmig.simple > pspi.simple.su
```

All of these programs are similar in structure, with only the interpolation algorithm being different. Each of these algorithms is easily extended to prestack application (with amplitudes not being preserved).

Note: All of these programs expect an input velocity in terms of interval velocities as a function of (x,z). Correspondingly, the output files of all of these programs are *depth sections*, which, if everything has been done correctly, is a representation of a cross-section through the earth.

2.0 Kirchhoff Migration of Zero-offset data

In 1978, Bill Schneider published his Kirchhoff migration method. The program that implements this in SU is called `sukdmig2d`.

There are two shell scripts in /data/Data2 that you will need to be able to run this on the simple.su data. Go to your home directory, make a temporary directory called Temp2, and copy

```
$ cd ESC
$ mkdir Temp2
$ cp /data/Data2/Rayt2d.simple Temp2
$ cp /data/Data2/Kdmig2d.simple Temp2
$ cp /data/Data2/simple.su Temp2
$ cp /data/Data2/vel.kdmig.simple Temp2
$ cd Temp2
```

Furthermore, you will need to make sure that the headers are correct on simple.su. You should see:

```
$ surange < simple.su
80 traces:
trac1 1 80 (1 - 80)
cdp 1 80 (1 - 80)
trid 1
sx 0 3160 (0 - 3160)
gx 0 3160 (0 - 3160)
ns 501
dt 4000
```

If not (for example, if the `sx` and `gx` fields are not set), then do the following

```
$ mv simple.su simple.orig.su
$ sushw < simple.orig.su key=sx,gx a=0,0 b=40,40 > simple.su
```

(If the source and geophone positions are not set, then the `sukdmig2d` program will think that all of the input data are at the position `sx=0, gx=0`.)

The method of migration is called “Kirchhoff” because the technique is based on an integral equation called the *Kirchhoff modeling formula*, which is a high-frequency representation of the wave field emanating from a reflector in the form of an integral over the reflector. Some people like to think of the Kirchhoff integral as describing an “exploding reflector” model of reflectivity. In some sense, the *Kirchhoff migration formula* is an approximate inverse of the Kirchhoff modeling formula, meaning that we are in an approximate sense solving for the reflectivity in the subsurface. The migration integral equation may be implemented as such, or as a sum (an integral) over diffractions (the diffraction stack). In either case, the *Kirchhoff migration formula may also be called a “Kirchhoff-WKBJ migration,”* where the WKBJ (Wentzel, Kramers, Brillouin, Jeffreys) denotes that ray theoretic quantities are being used. Most importantly travel times and some estimate of ray-theoretic amplitudes need to be computed for the background wave speed profile to do the migration. If you recall, we might view migration as moving data up or down along a light cone. Approximating the wave-field by ray tubes is one way of doing that.

The shell script Rayt2d.simple runs the program rayt2d, which generates the necessary traveltimes tables. The shell script Kdmig2d.simple runs sukdmig2d on the simple.su data. Read each shell script by typing:

```
$ more Rayt2d.simple
```

```
#!/bin/sh
```

```
rayt2d vfile=vel.kdmig.simple \
dt=.004 nt=501 fa=-80 na=80 \
fz=0 nz=501 dz=4 \
fx=0 nx=80 dx=40 ek=0 \
nxs=80 fxs=0 dxs=40 ms=1 \
tfile=tfile.simple
```

```
xmovie < tfile.simple clip=3 n1=501 n2=80 loop=1 title="%g traveltimes table"
&
```

You may type ‘q’ to get out of the more program.

The program rayt2d generates a traveltimes table containing the traveltimes from each source to each receiver for every point in the model. The movie shown by xmovie shows these traveltimes as shades of gray. The idea of running the movie over the traveltimes is to see if there are any inconsistencies in the collection of traveltimes.

```
$ more Kdmig2d.simple
```

```
#!/bin/sh
```

```
sukdmig2d infile=simple.su outfile=kdmig.simple.su ttfle=tfile.simple \  
ft=0 fzt=0 nzt=501 dzt=4.00 angmax=80.0 \  
fxt=0 nxt=80 dxt=40 fs=0 ns=80 ds=40 \  
dxm=40 v0=2000 noff=1 off0=0 doff=0
```

You may type 'q' to get out of the more program.

The program `sukdmig2d` performs the Kirchhoff migration drawing on the travel-times created by `rayt2d` that are in the file `tfile.simple`. Indeed, if a person had a better traveltime table generator, then this program could use those traveltimes instead.

To apply the Kirchhoff migration to the dataset `simple.su` we first type:

```
$ Rayt2d.simple
```

to generate the traveltime tables for `sukdmig2d`. You will notice that a little window showing a movie will appear. You may grab the lower corner of this window by clicking and dragging with your mouse to stretch the image to a larger size. This movie shows the map of traveltimes to each point in the subsurface from the common source-receiver position. As this is only a constant-background model, we would expect that the curves of constant traveltime (wavefronts) are circles. You should be able to detect this in the change of the grayscale. If the curves do not appear to be perfect circles, this is due the aspect ratio of the plot. You may stretch the plot so that it has the correct aspect ratio.

To perform the the Kirchhoff migration we type

```
$ Kdmig2d.simple
```

the resulting migrated data is the file `kdmig.simple.su` which you may view via

```
$ suximage < kdmig.simple.su &
```

3.0 Zero-offset $v(t)$ and $v(x,z)$ migration of real data

Now that you have had an introduction to a few zero-offset migration codes, we will apply some of these migrations to a dataset consisting of real data. As before, make a temporary directory in a scratch directory area. Call this one Temp3.

```
$ cd ESC
$ mkdir Temp3
$ cd Temp3
$ cp /data/Data3/Stoltmig .
$ cp /data/Data3/PSmig .
$ cp /data/Data3/seismic3.su .
$ cp /data/Data3/Suttoz.stolt .
$ cp /data/Data3/Suttoz.psmig .
```

These data were collected over a structure in the North Sea, known as the Viking Graben. The data were shot by Mobil Corporation back in the 1980s. This is a *stacked section*. The data seismic3.su have been gained, have had multiple suppression applied via supef (Wiener spiking and Wiener prediction error filter deconvolution), have been NMO corrected (velocity analysis performed with suvelan, and normal moveout corrected via sunmo), dip-moveout corrected with sudmofk, and have finally been stacked using sustack. The pre-processing is not very good. No migration has been applied.

First we check the header values on the data with surange

```
$ surange < seismic3.su 2142
traces:
trac1    1 2142 (1 - 2142)
tracr    1 120120 (1 - 120120)
fldr     3 1003 (3 - 1003)
tracf    1 120 (1 - 120)
ep       101 1112 (101 - 1112)

cdp      1 2142 (1 - 2142)
cdpt     1 120 (1 - 120)
trid     1
nhs      1 60 (1 - 1)
gelev    -10
selev    -6
scalel   1
scalco   1
sx       3237 28512 (3237 - 28512)
gx       0 28250 (0 - 28250)
```

```
counit  3
mute    48
ns      1500
dt      4000
```

It is convenient to copy and paste this screen into a file called “Notes” for future reference. We see that there are 2142 traces in the section, that the time sampling interval $dt = 4000$, which is to say $4ms$ sampling, and $ns = 1500$ samples per trace. We know from the headers on the original data that the spacing between midpoints, which is to say the spacing between CMPs, is $12.5m$.

You may view the data via:

```
suximage < seismic3.su perc=99 or
```

```
suximage < seismic3.su perc=99 verbose=1
```

The `verbose=1` option will show you the actual clip values that `perc=99` is giving you. You may see what the actual values of `bclip` (the numerical value of the color black) and `wclip` (the numerical value of the color white)

```
bclip=0.569718 wclip=-0.583124
```

You may then input clip values. For example

```
suximage < seismic3.su clip=.2 verbose=1
```

boosts the lower amplitudes. Try different values `clip=` to see what this does.

3.1 Stolt and Phaseshift $v(t)$ migrations

The easiest migrations are the Stolt and Phaseshift migrations. It is common to do a “quick look” at data by applying these types of migrations algorithms. Because a velocity analysis has been applied to these data, we have a collection of stacking velocities as a function of time, which we will make use of as an estimate of migration velocity.

If you type

```
$ sustolt
$ sumigps
```

and look carefully at the parameters for these programs, you will see that `sustolt` requires RMS velocities as a function of time, whereas `sumigps` requires interval velocities. If you look in the shell script `Stoltmig`, you will see

```
#!/bin/sh
sustolt < seismic3.su cdpmin=1 cdpmax=2142 \
tmig=0.0,1.0,2.5,3.45,4.36,5.1,5.45,5.95 \
vmig=1500,2000,3160,3210,3360,3408,3600,3650 \
```

```
dxcdp=12.5 > stolt.seis.su
```

```
exit 0
```

Here the pairs vmig= are RMS velocities (the velocities that come from velocity analysis—stacking velocities) for each time value in tmig=. These are only preliminary velocities taken from a single velocity analysis on the data.

If you look in the shell script PSmig, you will see

```
$ more PSmig
```

```
#!/bin/sh
```

```
sumigps < seismic3.su \
tmig=0.0,1.0,2.5,3.45,4.36,5.1,5.45,5.95 \
vmig=1500,2000,3738.45,3338,3876.32,3678.11,5706.7,4156.17 \
dx=12.5 > ps.seis.su
```

```
exit 0
```

Here the pairs vmig= are interval velocities (the actual seismic wavespeeds) for each time value in tmig=. The conversion of RMS to interval velocities is made through the application of the Dix equation. In SU, the program velconv is useful for a number types of velocities, including rms to interval velocities. The program suintvel is useful for converting a few stacking velocities to interval velocities.

For example, you can run suintvel to convert the stacking velocities as a function of time, used in the Stoltmig script into the interval velocities used in the PSmig script via

```
$ suintvel t0=0.0,1.0,2.5,3.45,4.36,5.1,5.45,5.95 \
          vs=1500,2000,3160,3210,3360,3408,3600,3650
```

The resulting velocities given by the value of v= below

```
h=0,1000,2803.84,1585.55,1763.72,1360.9,998.672,1039.04
v=1500,2000,3738.45,3338,3876.32,3678.11,5706.7,4156.17
```

The values of h= are the depths corresponding to the times in the t0= field. These could be used for model building, but are not used in migration. Again, these

values are only preliminary.

If you look inside the shell script `Suttoz.stolt`

```
$ more Suttoz.stolt
```

you will see that the same velocities are used for depth conversion from the Stolt migrated data (which is in time) to make a Stolt-migrated depth section.

Neither of these sets of velocities should be viewed as “exact”—these are only preliminary estimates. Notice, for example, that there is no lateral variation in these velocities.

These are only $v(t)$, which implies a $v(z)$, rather than a $v(x, z)$, profile. Yet, a cursory examination of the data shows a profile that dips to the right, indicating that there is likely substantial lateral variation of the velocities in the subsurface.

You may run the `Stoltmig` shell script by typing

```
$ Stoltmig
```

you will see that the output file is `stolt.seis.su`. You may create a depth section version of `stolt.seis.su` by typing:

```
$ Suttoz.stolt
```

The resulting output is the file `stolt.depth.seis.su`.

You may now plot all three of these files `seismic3.su`, `stolt.seis.su`, and `stolt.depth.seis.su` via

```
$ suximage < seismic3.su clip=.2 title="Stacked data" &  
$ suximage < stolt.seis.su clip=.2 title="Stolt time section" &  
$ suximage < stolt.depth.seis.su clip=.2 title="Stolt depth section" &
```

3.2 Phase Shift migration

We may now run the *phase shift migration* demo script `PSmig` by typing

```
$ PSmig
```

You will first notice first that it takes a bit longer to run the phase shift program than it did with `Stolt`. The advantage to phase shift migration is that propagates the field locally down in depth, so may handle the local variation of the background wavespeed better. The output of this program is a time section. You may convert this to a depth section by typing

```
$ Suttoz.psmig
```

As with the Stolt migration example, you may view the results via

```
$ suximage < seismic3.su clip=.2 title="Stacked data" &  
$ suximage < ps.seis.su clip=.2 title="PS time section" &  
$ suximage < ps.depth.seis.su clip=.2 title="PS depth section" &
```

Compile a summary report and submit your report.